Sprint 5

Auteurs : L3 CMI Informatique

- Angelo Adragna
- Corentin Saccoman
- Elliot Pozucek
- Colin Palazzetti Rubera
- Elvin Fichot-Corgeron

lien vers Github README Sprint 5

Rôles

- Angelo est `SCRUM Master` et `Product Owner`, il doit rédiger le rapport avec les différents artefacts ainsi qu'une ou deux pages synthétiques récapitulant le résultat du sprint.
- Corentin, Elliot, Colin et Elvin sont développeurs.
- Les développeurs travaillent par binômes.
- Une partie de l'équipe (Corentin et Elliot) doit passer le système sur les 10 articles pour détecter les limites fixes et verifier si le nombre de frontières veritables découpé par l'humain est identique à celui découpé automatiquement.
- Un second binôme (Colin et Elvin) devra s'occuper d'établir le corpus de référence (les frontières souples) des sections en analysant miniscieusement à la main chaque phrases et comptabiliser le nombre de points.

Objectif

Le Département de contrôle de qualité du LIA demande une évaluation de la qualité du système. Cette évaluation peut être réalisée en fonction de la précision obtenue sur un corpus de référence (découpage de sections fait par des humains). Ainsi, les fragments de texte extraits présents dans les références seront comptabilisés positivement; leur absence sera pénalisée. La précision sera calculée comme suit :

`Précision = Sections_correctes_trouvées_par_le_système / Sections_trouvées_par_le_système`

Une section sera considérée correcte si :

- a) Toutes ses phrases ont été bien détectées : frontières strictes.
- b) Quelques phrases peuvent chevaucher les sections (± 2 phrases) : frontières souples.

Il faut ajouter une dernière phase d'évaluation du système qui reporte le résultat de la précision. La précision sera mesurée sur un corpus de test de 10 nouveaux articles PDF. Une partie de l'équipe doit passer votre système sur les 10 articles, une autre doit établir le corpus de référence les frontières (souples) des sections. Enfin, vous devez calculer (à la main si vous voulez être agiles) la précision de votre système comme la moyenne sur l'ensemble des articles parsés.

Les détailles sont dans le sprint5.

Execution du programme

```
# Affichage de l'aide pour les options :
python parser.py -h

# Pas d'option de sélection des fichiers à parser :
python parser.py CORPUS_TRAIN/ CORPUS_TXT/                    # Format txt sans option
python parser.py -t CORPUS_TRAIN/ CORPUS_TXT/                 # Format txt avec option
python parser.py -x CORPUS_TRAIN/ CORPUS_XML/                 # Format xml
```

```
# Sélection manuelle des fichiers à parser :
python parser.py -m CORPUS_TRAIN/ CORPUS_TXT/                 # Format txt sans option
python parser.py -t -m CORPUS_TRAIN/ CORPUS_TXT/              # Format txt avec option
python parser.py -x -m CORPUS_TRAIN/ CORPUS_XML/              # Format xml

# Option pour ne pas afficher les informations de parsage
python parser.py --no-verbose CORPUS_TRAIN/ CORPUS_TXT/       # Format txt sans option sans les infos d
python parser.py -x --no-verbose CORPUS_TRAIN/ CORPUS_XML/    # Format xml sans les infos de parsage
python parser.py -x -m --no-verbose CORPUS_TRAIN/ CORPUS_XML/ # Séléction manuelle des fichiers à parse

# Format raw (sortie brute de pdftotext) pour le débug
python parser.py -r CORPUS_TRAIN/ CORPUS_RAW/
```

Résultats attendus :
Le répértoire CORPUS_TXT/ contiendra les fichiers texte avec uniquement le nom du fichier, le titre et l'abstract.
Le répértoire CORPUS_XML/ contiendra les fichiers xml avec toutes les infos demandées pour le xml.

Revu Sprint 5

- Debrief sur le produit du quatrième sprint mitigé. Le travail a été effecuté sans être testé, ceci à causé de gros problème à l'aube du sprint 5.
- Répartition des tâches par binômes selon les affinités de chacun. Communication très difficile.
- Les différents rendus des devs étaient satisfaisants.
- Un problème commun aux développeurs semble être le manque de robustesses du aux différements formats possibles au seins des PDF.

Réunion planification Sprint 16/05/25

La communication était mauvaise, le `SCRUM MASTER` à peiné à tenir notes de cette réunion car chacun est parti de son côté et que le contenu à réalisé est maigre et rapide à repartir.

Sprint Backlog

- Test précision avec des frontières strictes.
- Test précision avec des frontières souples.
- Calcul de la précision : Précision = Sections_correctes_trouvées_par_le_système / Sections_trouvées_par_le_système
- Améliorer les fonctionnalités du code.
- Réaliser les artefacts compris dans le conteneur de ce sprint 5.

Consignes pour le compte rendu des développeurs :

- Premier retour de la part des développeurs le dimanche 18/05/2025 sur le travail effecté.
- Déposez votre compte rendu au plus tard le ? à ? sur la branche 5.
- Fournir AU MOINS une image de l'avancement par fonctionnalité par développeur.
- Votre compte rendu doit comporter une analyse succinte de votre travail, tourné principalement autour de la `précision`… Vous pourrez vous baser sur le squelette pour vous guider si nécéssaire.
- Indiquer quel jour vous avez fini chaque étape de votre progression !
- Penser à bien décrire les différents calculs de précision de chaque document.

Travail de chaque membre de la `SCRUM Team` pendant la mêlée (Revue Sprint)

---

1. Angelo Adragna : Documentation des événements/artefacts Scrum (réunions de planification de sprint, mêlée quotidienne, revue de sprint, rétrospective du sprint) et rédaction condensé en 2 pages tournée vers la précision du programme.

- Les backlogs doivent être réalisés en premier lieu, lors de la première réunion, afin de répartir les tâches entre chaque personne.
- La réunion a permis d'établir la liste de questions pour le client, afin d'obtenir davantage de détails et de spécifications.
- Le Burndown se termine à la fin du sprint 5.
- La rédaction se fait au fil du sprint.
- Création d'un squelette de rendu pour facilier la mise en commun du travail de chacun. Je propose donc un modèle en raison du manque de détails dans les productions personnelles des développeurs.
- Médiation et conseils aux développeurs.
- Rédaction synthétique en une ou deux pages.

---

2. Elliot Pozucek, Corentin Saccoman : Test précision avec des frontières strictes (en binôme)

Développement

Comptage des frontières Afin d'évaluer la précision de notre algorithme sur l'ensemble de test, nous avons besoin de connaître le nombre de frontières présentes dans chaque article, le nombre de frontières trouvées par notre algorithme et le nombre de frontières trouvées par notre algorithme qui sont correctes (bien délimitées à +/- 2 lignes près).

Pour être précis :

- nous avons créé un fichier `frontieres.xml`
- ce fichier contient les mêmes balises que les fichiers xml résultants de l'algorithme (<titre>, <auteur>, etc)
- trois balises ont été ajoutées :
  - <frontieres-veritables> : le nombre de frontières réellement présentes dans l'article
  - <frontieres-trouvees> : le nombre de frontières trouvées par l'algorithme
  - <frontieres-correctes> : le nombre de frontières trouvées par l'algorithme qui sont correctes (bien délimitées à +/- 2 lignes près)
- Une balise a été ajoutée pour chaque article, avec un nom (ex: <document nom="article1">)

Comtage des frontières réelles Voici la méthode utilisée pour compter le nombre de frontières présentes dans un article :

- on éxécute l'algorithme avec l'option -r (python parser.py -r CORPUS_TEST/ CORPUS_TEST_RAW/)
- pour chaque article :
  - on ouvre le pdf et le fichier txt (raw)
  - pour chaque section :
    - on repère dans le pdf le début de la section
    - on cherche la première ligne de la section dans le txt (raw)
    - on écrit le numéro de la ligne dans le fichier `frontieres.xml`
    - on cherche la fin de la section dans le pdf
    - on cherche la première ligne de la section dans le txt (raw)
    - on écrit le numéro de la ligne dans le fichier `frontieres.xml`
    - cela donne par exemple : <titre>1-3</titre> (le titre se situe entre la ligne 1 et la ligne 3 dans le txt raw)
    - si la section n'existe pas dans le pdf, on écrit `None`

    - une fois ce travail effectué pour chaque section, on compte le nombre de sections réellement présentes dans l'article en comptant le nombre de balises où il n'y a pas de `None` et on écrit le résultat dans le fichier `frontieres.xml`, dans la balise <frontieres-veritables>
- une fois ce travail effectué pour chaque article, le fichier `frontieres.xml` contient le nombre de sections présentes dans chaque article

Comptage des frontières trouvées Voici la méthode utilisée pour compter le nombre de frontières trouvées par notre algorithme :

- on éxécute l'algorithme avec l'option -x (python parser.py -x CORPUS_TEST/ CORPUS_TEST_XML/)
- pour chaque article :
    - on ouvre le fichier xml
    - on compte le nombre de balises non-vides (la balise <preambule> est ignorée), le nombre de balises non-vides correspond au nombre de sections trouvées par l'algorithme
    - on écrit le résultat dans le fichier `frontieres.xml`, dans la balise <frontieres-trouvees>
- une fois ce travail effectué pour chaque article, le fichier `frontieres.xml` contient le nombre de sections trouvées par l'algorithme dans chaque article

Ajout de l'option –no-verbose (Elliot) Lors de la réalisation de ma partie du sprint-5, j'ai remarqué que le terminal était très pollué par les messages de parsing. Bien que ces messages soient utiles, j'ai décidé de créer une option `--no-verbose` pour permette de les désactiver.

J'ai donc ajouté la ligne suivante dans `parser.py`, dans la fonction `parse_arguments()` :

```
# Ajout d'un argument --no-verbose pour désactiver les messages d'information
parser.add_argument("--no-verbose", action="store_true", help="N'affiche pas les informations détaillées
```

J'ai ensuite ajouté une vérification dans les méthodes `convert_pdf_file()` et lors de l'énumération des fichiers pdf dans le `main` afin de ne pas afficher les messages si l'option `--no-verbose` est activée.

```
# convert_pdf_file
if not args.no_verbose :
    print(f"{fichier_pdf.name} converti en {fichier_sortie.name}")

# main
if not args.no_verbose:
    print("Fichiers PDF trouvés :")
    for pdf in pdfs:
        print(" -", pdf.name)
```

Cette option est robuste et fonctionne avec toutes les autres options (j'ai testé avec toutes les combinaisons d'options possibles).
Par exemple, on peut exécuter la commande suivante :

```
python parser.py -x --no-verbose CORPUS_TEST/ CORPUS_TEST_XML/
```

Cela va exécuter l'algorithme avec l'option -x (génération du fichier xml) et l'option `--no-verbose` (pas de messages d'information).

Ajout du fichier precision.py (Elliot) Une fois que le fichier `frontieres.xml` a été correctement rempli, j'ai voulu créer un script permettant de calculer la précision de notre algorithme en fonction de ce fichier.
J'ai donc créé le fichier `precision.py` qui contient la fonction `calcul_precision()`. Cette fonction prend en entrée un chemin (str) et écrit dans le terminal la précision de l'algorithme sur chaque article de l'ensemble de test, ainsi que la précision globale sur cet ensemble. Ce script ne fonctionne qu'avec le fichier `frontieres.xml` ou avec un fichier xml de même structure.

La précision pour chaque article est calculée conformément à la formule donnée dans l'Objectif du sprint-5 :

`precision = frontieres_correctes / frontieres_trouvees`

Le script va parser le fichier xml donné en entrée (`frontieres.xml`) et va, pour chaque article, récupérer le nombre de frontières trouvées et le nombre de frontières correctes. La précision est calculée puis affichée dans le terminal. Les erreurs sont gérées (ex: si le nombre de frontières trouvées est égal à 0, pas un nombre, si les balises ne sont pas présentes, etc).

Voici un exemple d'utilisation (précision = 0 car pas encore de comptage des frontières correctes) :

```
A_Benders_Decomposition_Approach_toCorrelation_Clustering.pdf                          : 0.00% (0/7)
A_memetic_algorithm_for_community_detectionin_signed_networks.pdf                      : 0.00% (0/7)
An_Improved_Branch-and-Cut_Code_for_the_Maximum_Balanced_Subgraph_of_a_Signed_Graph.pdf : 0.00% (0/6)
Cabrera_RESUMES_2019.pdf                                                               : 0.00% (0/8)
Conversational_Networks_for_Automatic_Online_Moderation.pdf                            : 0.00% (0/7)
Dynamical_Models_Explaining_Socia_Balance_and_Evolution_of_Cooperation.pdf             : 0.00% (0/7)
Exact_Clustering_via_Integer_Programming_and_Maximum_Satisfiability.pdf                : 0.00% (0/7)
LDA_resume.pdf                                                                         : 0.00% (0/8)
Partitioning_large_signed_two-mode_networks_Problems_and_prospects.pdf                 : 0.00% (0/7)
Polibits_42_02.pdf                                                                     : 0.00% (0/8)

Précision globale : 0.00%
```

Comptage manuel du score de chaque fichier (Corentin) Une fois le nombre de frontières déterminées, il a fallu vérifier manuellement leur correspondance avec la réalité.
J'ai alors rempli la balise <frontieres-correctes> afin d'apporter une consolidation aux résultats trouvés pas Colin et Elvin.

J'ai effectué cette mesure manuelle sur l'intégralité des fichiers de test.
Pour ce qui est des résultats trouvés par Elvin, ils concordaient.

Suite à ces mesures et au remplissage des balises du fichiers `frontieres.xml`, j'ai pu exécuter le programme `precision.py` afin de calculer la précision individuelle de chaque fichier ainsi que la précision globale du parseur.

```
A_Benders_Decomposition_Approach_toCorrelation_Clustering.pdf                          : 100.00% (7/7)
A_memetic_algorithm_for_community_detectionin_signed_networks.pdf                      : 28.57% (2/7)
An_Improved_Branch-and-Cut_Code_for_the_Maximum_Balanced_Subgraph_of_a_Signed_Graph.pdf : 66.67% (4/6)
Cabrera_RESUMES_2019.pdf                                                               : 62.50% (5/8)
Conversational_Networks_for_Automatic_Online_Moderation.pdf                            : 85.71% (6/7)
Dynamical_Models_Explaining_Socia_Balance_and_Evolution_of_Cooperation.pdf             : 42.86% (3/7)
Exact_Clustering_via_Integer_Programming_and_Maximum_Satisfiability.pdf                : 71.43% (5/7)
LDA_resume.pdf                                                                         : 75.00% (6/8)
Partitioning_large_signed_two-mode_networks_Problems_and_prospects.pdf                 : 57.14% (4/7)
Polibits_42_02.pdf                                                                     : 75.00% (6/8)

Précision globale : 66.49%
```

Le score global est fortement affecté par le deuxième fichier, ayant un formatage assez singulier, dégradant les performances du parseur.

Précision

On apperçoit que notre programme obtient un score de précision de 66.49% sur le jeu de test.

Piste d'amélioration

Le résultat n'est pas surprenant. Le programme est assez précis sur les documents qui respectent rigoureusement la structure établie sur les fichiers d'entrainement. Cependant, dès que cette structure varie, les performances en subissent les conséquences.
Il n'y a pas vraiment d'amélioration possible à apporter au programme. En effet, nous ne pouvons pas prédire comment un article de recherche va être rédigé, quel nom va avoir chaque partie etc…
Afin d'éviter de saisir des mots hors de leur contexte (par exemple de prendre un titre au milieu d'une phrase

à cause d'une numérotation), nous pourrions apporter une vérification sur le contexte du groupe de mots trouvé, afin d'augmenter la robustesse du programme.
Néanmoins, des sections comme celles des auteurs ou le titre, n'étant pas identifiées distinctement, demeurent impossibles à relever sans une part de hasard.

Dates clés :

- Corentin, Elliot, 16/05 : Réalisation du fichier `frontieres.xml` et découpage manuel des fichiers de test.
- Elliot, 18/05 : Réalisation du programme de test de précision, ajout d'un argument au parseur, actualisation du balisage de `frontieres.xml`
- Corentin, 18/05 : Détermination des performances du parseur et attente des résulats de Colin pour vérifications.

---

3. Colin Palazzetti Rubera : Test précision avec des frontières souples.

Retour sur les documents découpés

Je n'ai pas suivi le meme processus qu'Elvin, étant donné que je me suis remis à la tache après que Corentin et Elliot aient mis en ligne la première version des découpages, ainsi, j'ai pu ne vérifier que les frontières qui avaient été marquées comme incorrectes, considérant que la vérification étant brute, si le système disait correct, alors c'est correct en souple, dans le cas contraire, il faut vérifier si les opérations de modification des réponses n'ont pas altéré la vérification

Ainsi, je ne vérifie que si le programme a raison en disant que la frontières est incorrecte

A__Benders__Decomposition__Approach__toCorrelation__Clustering 7/7 frontières, je peux passer

A__memetic__algorithm__for__community__detectionin__signed__networks 2/7, à vérifier

- Titre
  - +5
  - non vérifiée
- Auteur
  - +7
  - non vérifiée
- Abstract
  - Correct
  - Correct
- Introduction
  - Non trouvée
- Corps
  - +42
  - non vérifiée
- Conclusion
  - Correct
  - Correct
- Discussion
  - Correct (Non trouvée)
- Biblio
  - Correct
  - Correct

J'ai trouvé une différence du nombre de frontières correctes, mais cela est probablement du au fait que la section biblio a été modifiée, et que la discussion n'existe pas

4/7 au lieu de 2/7

An_Improved_Branch-and-Cut_Code_for_the_Maximum_Balanced_Subgraph_of_a_Signed_Graph 4/6, à vérifier

- Titre
  - Correct
  - -1
- Auteur
  - -1
  - Correct
- Abstract
  - Correct
  - Incorrect (fin du document)
- Introduction
  - Correct
  - Correct
- Corps
  - Correct
  - Incorrect (fin du document)
- Conclusion
  - Correct (Non trouvée)
- Discussion
  - Correct (Non trouvée)
- Biblio
  - Correct
  - +1

4/6, confirmant le résultat du programme

Cabrera_RESUMES_2019 5/8, à vérifier

- Titre
  - +2
  - +2
- Auteur
  - -5
  - Non vérifiée
- Abstract
  - +14
  - Non vérifiée
- Introduction
  - Correct
  - Correct
- Corps
  - Correct
  - Correct
- Conclusion
  - Correct
  - Correct
- Discussion
  - +18
  - Non vérifiée

- Biblio
  - Correct
  - +1

  5/8, correct

Conversational_Networks_for_Automatic_Online_Moderation 6/7, à vérifier

- Titre
  - Correct
  - -1
- Auteur
  - -1
  - -1
- Abstract
  - Correct
  - Correct
- Introduction
  - Section non trouvée
- Corps
  - Correct
  - Correct
- Conclusion
  - Correct
  - Correct
- Discussion
  - Correct (non trouvée)
- Biblio
  - Correct
  - Correct

  7/8, correct, sachant que le programme n'a pas vérifié pour la discussion

Précision

Le programme créé par le reste de l'équipe vérifie bien les découpages, avec quelques différences mineures, qui sont dues au fait que certaines sections modifient le texte avant de le renvoyer (Biblio, auteur, abstract), ce qui rend plus difficile la recherche.

Piste d'amélioration

Il n'y a pas d'amélioration particulières sur le programme, mais une structure un peu plus universelle des rapports serait un plus pour ce genre d'outil. De plus, pour le peu de temps/ressources utilisées par ce programme, les résultats sont plus que satisfaisants.

Note sur le travail

Je n'ai pas été bon sur l'organisation du travail, je comptais m'occuper du travail sur mon dimanche, pour le rendu le dimanche suivant, mais vu que le rendu a été déplacé tardivement, l'équipe a du faire le travail en urgence, et je n'étais pas disponible (mon chat avait décidé de s'enfermer chez le voisin)

Ainsi, quand j'ai compris que le rendu avait été déplacée, j'ai du faire un peu de travail, chaque soir après mon stage, mais je n'ai pas été efficient, ni efficace.

---

4. Elvin Fichot-Corgeron : Test précision avec des frontières souples.

Le but de ma tâche est d'analyser les résultats trouvés par le parseur pour ensuite les comparer avec les frontières souples et ainsi calculer le score de précision.

Le score de précision se calcule avec la formule suivante :

$ Précision = nbsectioncorrectes/sectiontrouvé$

Pour rappel, on considére qu'une section est correcte si elle est dans la frontière souple (au plus 2 lignes d'écart avec la frontière stricte)

Dynamical_Models_Explaining_Socia_Balance_and_Evolution_of_Cooperation

Frontières souples

- Titre : L1-2 valide
- Auteur : L2-7 + L19-~32 non valide
- Abstract : L8- ~32 non valide
- Introduction : L33- ~607 non valide
- Corps : L_{608-}780 non valide
- Conclusion : L507-556 valide
- Discussion : Il n'y a pas de discussion
- Biblio : L790-892 valide

Précision Il y a au total 7 sections et nous en trouvons 3 de correctes.

Le taux de précision est donc de $3/7 = 0.43$

Exact_Clustering_via_Integer_Programming_and_Maximum_Satisfiability

Frontières souples

- Titre : L1-2 : valide
- Auteur : L3-14 valide recupère bien toutes les informations même si certaines sont dupliqués
- Abstract : L16-37 valide
- Introduction : L39-172 non valide
- Corps : L173-922 non valide
- Conclusion : L924-964 valide (souple)
- Discussion : Pas de discussion dans le document
- Biblio : L971-1063 valide (souple)

Précision Il y a au total 7 sections et nous en trouvons 5 de correctes.

Le taux de précision est donc de $5/7 = 0.71$

LDA_resume

Frontières souples

- Titre : L1-2 valide
- Auteur : L3-18 non valide il manque plusieurs lignes qui ne sont pas détecter
- Abstract : L20-37 valide
- Introduction : L39-104 valide
- Corps : L105-796 non valide
- Conclusion : L798-811 valide
- Discussion : L666-796 valide
- Biblio : L813-877 valide

Précision Il y a au total 8 sections et nous en trouvons 6 de correctes.

Le taux de précision est donc de $6/8 = 0.75$

Partitioning__large__signed__two-mode__networks__Problems__and__prospects

Frontières trouvés par le parseur

- Titre : en tête de document non valide
- Auteur : L1 non valide
- Abstract : L3 non valide
- Introduction : L36-117 valide
- Corps : L118-1739 valide
- Conclusion : Pas de conclusion dans le document
- Discussion : L1740-1819 valide
- Biblio : L2069-2192 valide (souple)

Précision Il y a au total 7 sections et nous en trouvons 4 de correctes.

Le taux de précision est donc de $4/7 = 0.57$

Polibits__42__02

Frontières souples

- Titre : L1-2 : valide
- Auteur : L3 : valide
- Abstract : L4-18 : valide
- Introduction : L33-82 valide
- Corps : L135-552 : non valide
- Conclusion : L554-566 : non valide
- Discussion : L512-552 : valide
- Biblio : L661-746 : valide

Précision Il y doit y avoir en tout 8 sections nous en trouvons 6 de correctes.

Le taux de précision est donc de $6/8 = 0.75$

Remarque

Pour ce sprint j'ai rencontré une petite difficulté notament pour évaluer les auteurs. En effet il avait été convenu au sprint 2 de noter les auteurs de la manière suivante : *nom adresses* (universitaire et mail si précisés) Cependant, cette notation n'était pas en accord avec la manière de déterminer si une section est correcte. J'ai donc regardé bien en détails les lignes pour voir si les informations étaient plutôt cohérentes et essayé de me rapprocher au plus de la marge de souplesse déterminée.

Lorsque que j'ai noté les ~ pour les lignes c'est que la marge d'erreur est bien plus grande que 2 lignes et que cela ne changera pas le résultat.

Retrospective sur le projet en entier

De mon point de vue, j'ai trouvé le projet très intéressant. Il démontre clairement la nécessité pour un développeur d'avoir un code robuste, car le projet peut évoluer à n'importe quel moment. Pour ce qui est de la méthode Scrum, j'ai déjà eu l'occasion de travailler avec aussi bien lors de plusieurs projets dans mon ancienne formation ou bien aussi en entreprise lors de mon alternance.

Retour sur le travail effectué (Rétrospective Sprint 5)

Lors de ce cinquième sprint, nous avons poursuivi le développement et l'amélioration de notre parseur de documents scientifiques. Les objectifs principaux étaient :

- L'extraction précise des sections `Introduction` et `Corps` à partir de différents formats de fichiers (PDF, TXT, XML).
- La mise en place de tests de précision pour évaluer la performance du parseur.
- L'amélioration de l'interface utilisateur pour une sélection interactive des fichiers à traiter.
- Le nettoyage et la restructuration du code existant pour le rendre plus maintenable.

Chaque membre de l'équipe a pris en charge un aspect spécifique du sprint, ce qui nous a permis de progresser efficacement en parallèle. Une attention particulière a été portée à la gestion des cas particuliers rencontrés dans les formats hétérogènes des documents. Le suivi régulier des tâches en daily meetings a également contribué à une meilleure répartition de la charge de travail et à une communication fluide.

Conclusion :

Selon Angelo Adragna

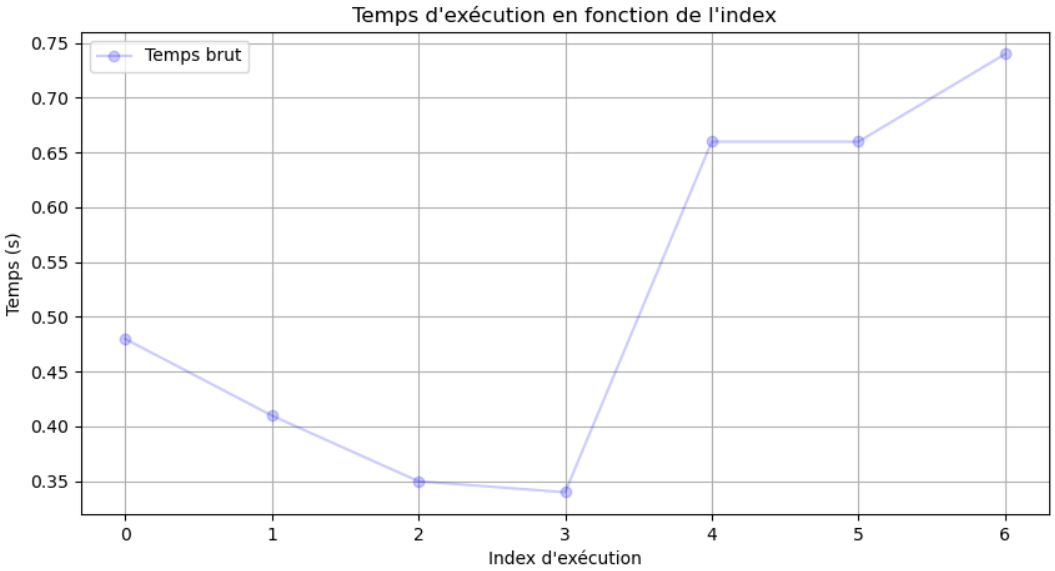
Durant ce sprint, j'ai travaillé sur l'intégration des expressions régulières et l'extraction des sections ciblées dans les documents XML. Le découpage des tâches était clair et m'a permis de rester concentré. Quelques ajustements ont été nécessaires sur l'uniformisation des balises. Pour la suite, je compte améliorer l'anticipation des cas limites et des tests.

Selon Corentin Saccoman

Je me suis concentré sur le menu de sélection interactif et son adaptation aux différents types de documents (TXT, PDF, XML). Ce sprint s'est bien déroulé en termes de coordination. J'ai néanmoins remarqué que la phase de test arrive souvent trop tard dans notre cycle de développement. Je proposerai d'intégrer des tests progressifs au fil du sprint.

Selon Elliot Pozucek

J'étais en charge de la mise en place des tests de précision sur les résultats du parseur. Cela m'a permis d'évaluer objectivement la qualité de l'extraction selon le type de document.



L'image ci-dessus illustre un exemple des écarts de temps détectés entre les sections extraites et les résultats attendus. Ce travail m'a aidé à comprendre les limites actuelles du système et les pistes d'optimisation.

Selon Colin Palazzetti Rubera

J'ai principalement retravaillé le cœur du parseur en refactorant certaines fonctions et en améliorant leur modularité. J'ai aussi veillé à ce que notre code respecte une meilleure organisation. Le projet est devenu plus lisible et maintenable. Il reste des efforts à faire sur la génération automatique de la documentation technique et les tests à l'intégration.

Global

Ce sprint a marqué un tournant positif dans notre organisation. La répartition des rôles était bien équilibrée et nous avons su avancer de manière coordonnée malgré la complexité croissante du projet. Les livrables sont fonctionnels, mais des améliorations sont encore nécessaires sur la robustesse des tests, l'uniformité des formats de sortie, et la documentation. Nous allons aborder le sprint suivant avec ces objectifs en tête.